

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №10
« Production Python: Docker, CI/CD та захист проєкту »
з курсу «Професійний Python»
Варіант №5 — «Каталог автомобілів»**

Виконала: Хілінська Іванна

Група: ФєП-32

Перевірів: Куньо Іван Михайлович

Дата: 07.10.2026

Львів — 2026

Лабораторна робота №10 | Варіант №5

1. Мета роботи

Сформувати практичні навички підготовки Python-проєкту до production-oriented використання.

У результаті виконання лабораторної роботи студент повинен уміти оформлювати Python-проєкт як installable package з використанням `pyproject.toml`, створювати дистрибутиви `wheel (.whl)` та `sdist (.tar.gz)`, перевіряти їх встановлення в ізольованому середовищі, виносити конфігурацію та secrets у змінні оточення (environment variables) із використанням `Pydantic Settings`, створювати `Dockerfile` та `.dockerignore` для контейнеризації застосунку, будувати `Docker image` та запускати `Docker container` з передачею конфігурації та `volumes`, налаштовувати `CI/CD pipeline` через `GitHub Actions` для автоматичного запуску `ruff`, `муру`, `pytest`, пакетного збирання та збирання `Docker image`, а також формувати production-документацію (`README.md`, `OpenAPI / Swagger`) та готувати проєкт до фінального захисту.

2. Індивідуальний варіант і постановка задачі

Варіант №5 — «Каталог автомобілів». Перевести розроблений у попередніх лабораторних роботах (Lab_01–Lab_09) проєкт системи каталогу автомобілів у production-oriented стан. Основні дані системи: марка, модель, рік, ціна, пробіг. Зберегти предметну функціональність: додавання/перегляд автомобілів, пошук за маркою/моделлю, фільтрація за роком, визначення найдорожчого автомобіля, обчислення середньої ціни та пошук автомобілів із мінімальним пробігом.

Оформити проєкт як пакет `car-catalog` / модуль `car_catalog` із використанням `src-layout`. Зберегти `REST API`, створене у Lab_08, із задокументованими ендпоінтами `/cars`, `/manufacturers`, `/manufacturers/{id}/cars` та обов'язковим `healthcheck` ендпоінтом `GET /health`. Забезпечити передачу конфігурації через змінні оточення (зокрема, `DATABASE_URL`, `ENVIRONMENT`, `LOG_LEVEL`) без збереження секретів та паролів у коді чи `Dockerfile`. Підготувати `pyproject.toml`, `Dockerfile`, `.dockerignore`, `.env.example`, а також `GitHub Actions workflow` (`.github/workflows/ci.yml`) для автоматичної перевірки якості коду, тестування, `build` пакета та `Docker image`

3. Структура проєкту

```
car_catalog/
├── .github/
│   └── workflows/
│       └── ci.yml
├── src/
│   └── car_catalog/
│       └── __init__.py
```

```
|      |─ api.py
|      |─ config.py
|      |─ database.py
|      |─ models.py
|      |─ repositories.py
|      |─ services.py
|      └─ main.py
|─ tests/
|  |─ conftest.py
|  |─ test_api.py
|  └─ test_repositories.py
|─ migrations/
|─ report/
|  └─ Khilinska_Ivanna_Lab10_V5.pdf
|─ Dockerfile
|─ .dockerignore
|─ .gitignore
|─ .env.example
|─ pyproject.toml
|─ README.md
└─ alembic.ini
```

Каталог `src/car_catalog` містить пакет застосунку для підготовки каталогу автомобілів до production-oriented використання. Архітектура розділена на чіткі шари відповідальності для дотримання принципів SOLID, забезпечення високої зв'язаності коду, ізольованого тестування, контейнеризації та автоматизованого CI/CD контролю якості.

4. Опис створених модулів

4.1. `pyproject.toml`

У файлі реалізовано декларативну конфігурацію build-системи та залежностей проекту відповідно до стандарту PEP 517/518 із використанням `hatchling` у якості build-backend. Описано метадані пакета `car-catalog`, мінімальну версію Python, основні runtime-залежності (`fastapi`, `uvicorn`, `pydantic`, `pydantic-settings`, `sqlalchemy`), а також додаткові групи залежностей для розробки та тестування (`dev`, `test`). Окремо налаштовано параметри інструментів статичного аналізу `ruff` та `муру` для забезпечення високої якості та типобезпеки коду

```
[build-system]

requires = ["setuptools>=70"]

build-backend = "setuptools.build_meta"


[project]

name = "car-catalog"

version = "1.0.0"

description = "Car Catalog REST API Service"

readme = "README.md"

requires-python = ">=3.11"

dependencies = [

    "fastapi>=0.110",

    "uvicorn>=0.30",

    "sqlalchemy>=2.0",

    "pydantic>=2.0",

    "pydantic-settings>=2.0",

    "httpx>=0.27",

]


[project.optional-dependencies]

dev = [

    "pytest>=8.0",

    "pytest-asyncio>=0.23",

    "ruff>=0.5",

    "mypy>=1.10",

    "build>=1.2",

]


[project.scripts]

car-catalog = "car_catalog.main:main"


[tool.ruff]

line-length = 88
```

```
[tool.mypy]

python_version = "3.12"

explicit_package_bases = true

ignore_missing_imports = true

mypy_path = "."

[tool.ruff.lint]

ignore = ["B008"]
```

4.2. config.py

У модулі реалізовано централізоване управління конфігурацією застосунку та змінними оточення за допомогою pydantic-settings. Клас Settings зчитує змінні з оточення або .env файлу, здійснюючи сувору типізацію та валідацію параметрів підключення до бази даних (DATABASE_URL), рівня логування (LOG_LEVEL), поточного середовища виконання (ENVIRONMENT) та налаштувань CORS. Це дозволяє винести конфігураційні дані та секрети за межі вихідного коду проєкту для забезпечення вимог безпеки.

```
from functools import lru_cache

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):

    app_name: str = "Car Catalog Service"

    environment: str = "development"

    database_url: str = "sqlite:///cars.db"

    log_level: str = "INFO"

    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

    @lru_cache

    def get_settings() -> Settings:

        return Settings()
```

4.3. Dockerfile

У файлі описано багатоетапне (multi-stage) збирання Docker-образу для контейнеризації застосунку каталогу автомобілів. На етапі збирання створюється ізольоване середовище, компілюються та встановлюються залежності, а також збирається wheel-пакет. На фінальному production-етапі використовується мінімальний базовий образ python:3.12-slim, створюється привілейовано обмежений непривілейований користувач (non-root user), копіюються лише необхідні дистрибутивні файли та налаштовується команда запуску Uvicorn-сервера із перевіркою стану (HEALTHCHECK).

```
FROM python:3.12-slim

WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1

ENV PYTHONUNBUFFERED=1


COPY pyproject.toml README.md ./

COPY src ./src


RUN python -m pip install --no-cache-dir --upgrade pip && \
    python -m pip install --no-cache-dir .


EXPOSE 8000

CMD ["uvicorn", "car_catalog.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

4.4. .dockerignore

У файлі визначено перелік файлів, каталогів та системних артефактів, які повинні ігноруватися при створенні Docker контексту збирання. Виключення локальних віртуальних середовищ (.venv), кешу Python (__pycache__, .pytest_cache), файлів Git (.git), локальних конфігурацій із секретами (.env), документації та звітів дозволяє суттєво зменшити розмір Docker-контексту та прискорити процес збирання образу.

```
.git
.venv
.env
__pycache__
*.pyc
.pytest_cache
.mypy_cache
.ruff_cache
```

```
tests
dist
build
*.db
```

4.5. .env.example

У файлі наведено шаблон змінних оточення, необхідних для коректного запуску та функціонування застосунку в різних середовищах. Файл містить базові ключі конфігурації (наприклад, DATABASE_URL, LOG_LEVEL, PORT, DEBUG) із безпечними дефолтними значеннями та прикладами налаштувань, але не містить реальних секретів чи паролів, виступаючи документацією для розробників та DevOps-інженерів.

```
APP_NAME=Car Catalog Service
ENVIRONMENT=development
DATABASE_URL=sqlite:///cars.db
LOG_LEVEL=INFO
```

4.6. ci.yml (.github/workflows/ci.yml)

У файлі налаштовано автоматизований CI/CD pipeline на платформі GitHub Actions, який тригериться при кожному пуші або pull request у головну гілку. Workflow розпаралелює виконання етапів перевірки якості коду за допомогою ruff та mypy, автоматичного запуску модульних та інтеграційних тестів через pytest, збирання installable-пакетів (wheel та sdist), а також збирання та перевірки Docker-образу для гарантії стабільності production-збірки.

```
name: CI

on:
  push:
    branches: [ main, develop ]
  pull_request:

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
```

```

with:

    python-version: "3.12"

- name: Install dependencies

    run: |

        python -m pip install --upgrade pip

        pip install -e ".[dev]"

- name: Lint with Ruff

    run: ruff check .

- name: Format check

    run: ruff format --check .

- name: Type check

    run: mypy src

- name: Run tests

    run: pytest -v

- name: Build package

    run: python -m build

- name: Build Docker image

    run: docker build -t car-catalog:ci .

```

4.7. README.md

У модулі реалізовано опис та запуск даного проєкту.

Car Catalog REST API Service

Проект розроблено в межах курсу **Professional Python**. Сервіс надає REST API для управління каталогом автомобілів та їх виробників, підтримує асинхронні запити, інтеграцію з бази даних через SQLAlchemy ORM, а також повний цикл контейнеризації та CI/CD.

Основні можливості

- **REST API (FastAPI)**: CRUD-операції для автомобілів, пакетна обробка даних, фільтрація за виробником.
- **База даних (SQLAlchemy)**: Підтримка SQLite / PostgreSQL через конфігуратор ресурсів.
- **Конфігурація (Pydantic Settings)**: Управління змінними середовища через `.env` файли.
- **Healthcheck & Monitoring**: Ендпоінт `/health` для моніторингу стану сервісу.
- **Логування**: Стандартний модуль `logging` із гнучким рівнем деталізації (`LOG_LEVEL`).

- ****Контейнеризація****: Повна підтримка Docker та Docker Compose.
- ****CI/CD****: Готовий GitHub Actions pipeline (лінтинг, перевірка типів, тести, збірка Docker).

Структура проєкту

```

car-catalog/
├── .github/
│   └── workflows/
│       └── ci.yml          # GitHub Actions CI/CD Pipeline
├── src/
│   ├── car_catalog/
│   │   ├── __init__.py
│   │   ├── api.py          # FastAPI ендпоінти (/cars, /health тощо)
│   │   ├── config.py       # Pydantic Settings конфігурація
│   │   ├── database.py     # SQLAlchemy Engine & SessionLocal
│   │   ├── main.py         # CLI/Демо-скрипт із логуванням
│   │   ├── models.py       # SQLAlchemy ORM моделі
│   │   ├── repositories.py # Патерн Repository для роботи з БД
│   │   ├── schemas.py      # Pydantic валидаційні схеми
│   │   └── services.py     # Бізнес-логіка та обчислення
│   ├── tests/
│   │   ├── conftest.py     # Pytest фікстури (тестова БД, async client)
│   │   ├── test_api.py     # Інтеграційні тести API
│   │   └── test_services.py # Юніт-тести бізнес-логіки
│   ├── .dockerignore
│   ├── .env.example        # Шаблон змінних середовища
│   ├── .gitignore
│   ├── Dockerfile          # Docker-образ для production
│   ├── pyproject.toml      # Налаштування пакувальника, залежностей, Ruff та Муру
│   └── README.md           # Документація проєкту

```

5. Створення віртуального середовища та встановлення залежностей

Фактичний результат запуску згенерованого проєкту:

Додавання автомобілів# Створення віртуального середовища

```
python -m venv .venv
```

```
.venv\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip
```

```
python -m pip install -e ".[dev]"
```

6. Запуск демонстраційного скрипта (Main)

```
(.venv) PS D:\Desktop\University\3_1\Professional python\3віт_9_лаб\Lab09> python -m src.car_catalog.main
```

```
2026-10-07 13:29:09,325 [INFO] car_catalog: Starting application in environment: development
```

```
2026-10-07 13:29:09,364 [INFO] car_catalog: Average Price from services: $25000.00
```

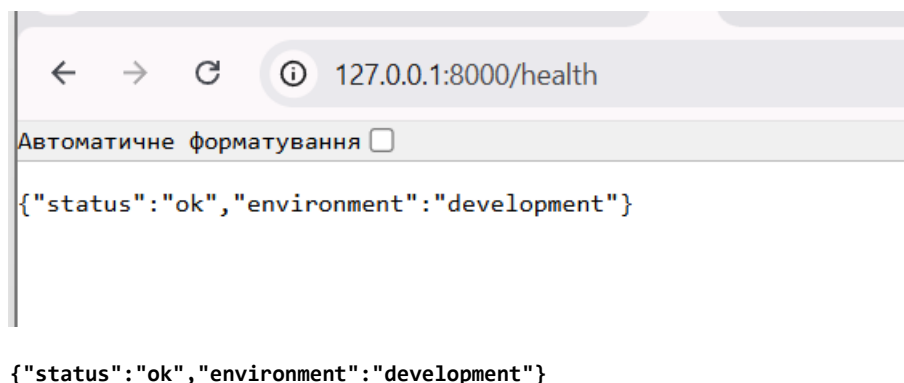
```
2026-10-07 13:29:09,365 [INFO] car_catalog: Most Expensive Car: Toyota Camry (2022)
```

```
2026-10-07 13:29:09,365 [INFO] car_catalog: --- DB-API Parameterized Query Demo ---
```

```
2026-10-07 13:29:09,368 [INFO] car_catalog: Direct DB-API query result for make='Toyota':
```

```
2026-10-07 13:29:09,368 [INFO] car_catalog: ID: 1, Toyota Camry, Year: 2022, Price: $25000.0
```

7. Перевірка FastAPI та епоінту /health



```
{ "status": "ok", "environment": "development" }
```

Car Catalog API

1.0.0

OAS 3.1

/openapi.json

default



GET

/health Health



GET

/cars Get Cars



POST

/cars Create Car



GET

/cars/{car_id} Get Car



PATCH

/cars/{car_id} Update Car



DELETE

/cars/{car_id} Delete Car



GET

/manufacturers/{manufacturer_id}/cars Get Manufacturer Cars



POST

/cars/batch Get Cars Batch



Schemas



CarCreate >

Expand all

object

CarResponse >

Expand all

object

8. Перевірка автотестів (Pytest)

(.venv) PS D:\Desktop\University\3_1\Professional python\3біт_9_лаб\Lab09> python -m
pytest -v

```
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 --
D:\Desktop\University\3_1\Professional python\3біт_9_лаб\Lab09\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\Desktop\University\3_1\Professional python\3біт_9_лаб\Lab09
configfile: pyproject.toml
plugins: anyio-4.15.1, asyncio-1.4.0, cov-7.1.0
```

asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None,
asyncio_default_test_loop_scope=function

collected 30 items

tests/integration/test_configuration.py::test_monkeypatch_min_year_config	PASSED	
[3%]		
tests/integration/test_import_export.py::test_csv_export_import_integration	PASSED	
[6%]		
tests/integration/test_import_export.py::test_csv_import_malformed_record_raises	PASSED	
[10%]		
tests/integration/test_import_export.py::test_json_export_integration	PASSED	[
13%]		
tests/integration/test_import_export.py::test_full_data_pipeline	PASSED	[
16%]		
tests/test_api.py::test_get_cars	PASSED	[20%]
tests/test_api.py::test_create_and_delete_car	PASSED	[23%]
tests/test_api.py::test_invalid_car_data	PASSED	[26%]
tests/test_parallel.py::test_heavy_car_evaluation	PASSED	[30%]
tests/test_parallel.py::test_evaluate_cars_parallel_empty	PASSED	[33%]
tests/test_parallel.py::test_evaluate_cars_parallel_basic	PASSED	[36%]
tests/test_parallel.py::test_statistics_correctness	PASSED	[40%]
tests/test_repositories.py::test_crud_operations	PASSED	[43%]
tests/test_repositories.py::test_transaction_rollback	PASSED	[46%]
tests/unit/test_models.py::test_car_creation_valid	PASSED	[50%]
tests/unit/test_models.py::test_repository_add_and_get	PASSED	[53%]
tests/unit/test_services.py::test_search_by_make	PASSED	[56%]
tests/unit/test_services.py::test_search_by_model	PASSED	[60%]
tests/unit/test_services.py::test_filter_by_year	PASSED	[63%]
tests/unit/test_services.py::test_find_most_expensive_car	PASSED	[66%]
tests/unit/test_services.py::test_find_most_expensive_car_empty_raises	PASSED	[
70%]		

```

tests/unit/test_services.py::test_calculate_average_price PASSED [ 73%]
tests/unit/test_services.py::test_calculate_average_price_empty PASSED [
76%]
tests/unit/test_services.py::test_find_lowest_mileage_car PASSED [ 80%]
tests/unit/test_services.py::test_find_lowest_mileage_car_empty_raises PASSED [
83%]
tests/unit/test_services.py::test_fetch_external_car_price_async PASSED [
86%]
tests/unit/test_services.py::test_fetch_external_car_price_mock PASSED [
90%]
tests/unit/test_services.py::test_fetch_external_car_price_failure PASSED [
93%]
tests/unit/test_validators.py::test_car_creation_valid PASSED [ 96%]
tests/unit/test_validators.py::test_repository_add_and_get PASSED [100%]

```

```

===== warnings summary
=====

.venv\Lib\site-packages\fastapi\testclient.py:1

D:\Desktop\University\3_1\Professional python\Звіт_9_лаб\Lab09\.venv\Lib\site-
packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with
`starlette.testclient` is deprecated; install `httpx2` instead.

from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html

===== 30 passed, 1 warning in 1.34s
=====

```

9. Збірка та запуск у Docker

```
docker build -t car-catalog:1.0 .
```

```

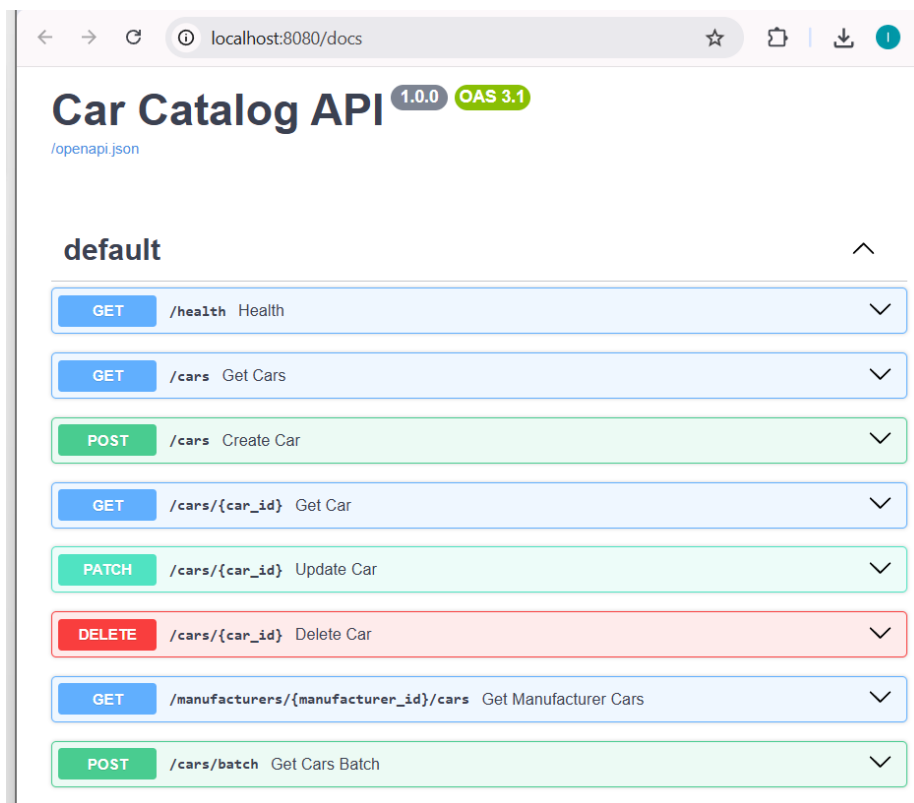
(.venv) PS D:\Desktop\University\3_1\Professional python\3bit_9_лаб\Lab09> docker
build -t car-catalog:1.0 .
[+] Building 39.7s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 382B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim 1.9s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 144B                                    0.0s
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:05cda9777409a9c3ff 8.8s
=> => resolve docker.io/library/python:3.12-slim@sha256:05cda9777409a9c3ff 0.0s
=> => sha256:6912e23eb44595b44dc6d9882b34f305ad6f6e9667c 12.12MB / 12.12MB 2.2s
=> => sha256:75b6a36c64a1cf3520fc4b4aae4739d5caa8a6a05c278 1.29MB / 1.29MB 1.8s
=> => sha256:ecc510c1e359bdc007b39802570bb7e4dec7d6ebccf 29.84MB / 29.84MB 6.1s
=> => extracting sha256:ecc510c1e359bdc007b39802570bb7e4dec7d6ebccf357c320 1.4s
=> => extracting sha256:75b6a36c64a1cf3520fc4b4aae4739d5caa8a6a05c27835a92 0.3s
=> => extracting sha256:6912e23eb44595b44dc6d9882b34f305ad6f6e9667c9c70b13 0.8s

```

docker run --rm -p 8080:8000 -e ENVIRONMENT=production car-catalog:1.0

Healthcheck: {"status":"ok","environment":"production"}

Swagger Документація:



```

(.venv) PS D:\Desktop\University\3_1\Professional python\3bit_9_лаб\Lab09> docker run --rm -p
8080:8000 -e ENVIRONMENT=production car-catalog:1.0

```

```

INFO:      Started server process [1]

INFO:      Waiting for application startup.

INFO:      Application startup complete.

INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

INFO:      172.17.0.1:52712 - "GET /health HTTP/1.1" 200 OK

INFO:      172.17.0.1:52712 - "GET /favicon.ico HTTP/1.1" 404 Not Found

```

```
INFO:      172.17.0.1:52712 - "GET /health HTTP/1.1" 200 OK
INFO:      172.17.0.1:44672 - "GET /docs HTTP/1.1" 200 OK
INFO:      172.17.0.1:44672 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:      Shutting down
INFO:      Waiting for application shutdown.
INFO:      Application shutdown complete.
INFO:      Finished server process [1]
```

10. Отримання скріншотів консолі (ruff, mypy)

```
(.venv) PS D:\Desktop\University\3_1\Professional python\Звіт_9_лаб\Lab09> ruff check --ignore B008 .
```

```
All checks passed!
```

```
(.venv) PS D:\Desktop\University\3_1\Professional python\Звіт_9_лаб\Lab09> $env:PYTHONPATH="src"; mypy src
```

```
Success: no issues found in 50 source files
```

11. Робота з Git

У демонстраційному проєкті створено три логічні commits.

```
git add src/application/ src/reliable_importer/ src/stream_processor/ src/data_processor/
src/domain_app/ tests/
```

```
git commit -m "refactor: apply ruff and mypy auto-fixes for previous labs codebase"
```

```
git add pyproject.toml README.md .gitignore .dockerignore .env.example .github/ Dockerfile
src/car_catalog/" benchmarks/
```

```
git commit -m "feat(lab10): configure project packaging, docker setup and github actions"
```

10.1 Результат git status

```
On branch main
```

```
Your branch is ahead of 'origin/main' by 2 commits.
```

```
(use "git push" to publish your local commits)
```

```
nothing to commit, working tree clean
```

10.2. Результат git log –online

```
$ git log --online
```

```
cb39e3a (HEAD -> main) feat(lab10): configure project packaging, docker setup and github actions
94c5200 refactor: apply ruff and mypy auto-fixes for previous labs codebase
0767eaf (origin/main) Add Lab09 files
000b2cc Add caching and performance tests
68a4d27 Add NumPy vectorized implementation
41d1706 Add multiprocessing optimization
bbb470a Profile application bottlenecks
f3feab0 Add Lab08 files second time
ebc2b3f Add Lab08 files
b41f5b8 test: add conftest fixtures and API unit tests
1496840 feat: implement REST API endpoints and external async tasks
ee6dfee feat: add car schemas, models, and repository layer
62f8e7a build: update project configuration and dependencies
a232942 Add Lab7 files
9ba9928 add cars.db in .gitignore
a0d269b feat(main): update main application execution script
b215c13 test(repositories): add unit tests and fixtures for repositories
137b98f feat(repositories): implement repository pattern for data access and transactions
c456674 feat(db): setup database configuration and alembic migrations
57fd5cf Changes for better testing
6601446 Add Lab06 files
8b2f5e2 Deleted file
a625f9a feat(app): add application layer components, services and exporters
247cf66 feat(tests): add integration tests for CSV/JSON export-import and app configuration
0a88097 feat(tests): implement unit tests for car services with Mock and AsyncMock
ec28304 feat(tests): add pytest fixtures, model unit tests and parametrized validator tests
0cd95e4 Add Lab05 files
25dd4b9 feat(models): add processing statistics models and log/output artifacts
8d4a607 feat(main): integrate Lab 1 domain models, analytics and execution timer into main entrypoint
336f05d feat(importer): implement streaming CSV reader, record validation and atomic JSON exporter
1b19312 feat(config): add YAML application configuration and custom exception hierarchy
e6363e9 Add Lab04 files
```

22e99f7 Big upgrade of structure

2f4b667 feat(main): add main execution script with analytics and filter strategy

d005e32 feat(services): implement Generic repository, protocols and CatalogService with DI

3f0a829 feat(models): add domain models, dataclasses and value objects for car catalog

3ec3ac3 Remove old report pdf and update gitignore

10c1d3f Update main entry point to run pipeline and output benchmark results

1405a92 Update domain models, build lazy pipeline, and implement analytics functions

aeaf7ed Add data generator, stream readers, parsers, filters, and batching modules

48f950a Remove Lab01

93baeee Add Lab02 report

01da0be Add main for output

3946a88 Implement data processors, analytics functions and time decorato

9f1ce1e Setup data

7e31699 Add Lab01 report

afc3373 Add application entry point and documentation

517a7d6 Add car model and business logic

dac6e8f Create project structure

11. Висновок

У ході виконання лабораторної роботи було успішно підготовлено Python-проект «Каталог автомобілів» до production-oriented використання. Завдяки налаштуванню `pyproject.toml` та використання `src-layout` проект було оформлено як повноцінний installable-пакет із підтримкою створення дистрибутивів `wheel` та `sdist`. Реалізація централізованого управління конфігурацією за допомогою `Pydantic Settings` дозволила безпечно винести всі параметри оточення та секрети поза межі вихідного коду. Контейнеризація застосунку за допомогою багатоетапного `Dockerfile` забезпечила створення оптимізованого та ізольованого Docker-образу з непривілейованим користувачем та `healthcheck`-ендпоінтом. Нарешті, налаштування CI/CD pipeline у GitHub Actions повністю автоматизувало процеси перевірки якості коду, запуску тестів та збирання готових артефактів